Authentication System Documentation

Express.js + MongoDB + JWT + Google Login (Without Passport.js)

Overview

This authentication system supports:

- Email Registration
- Email Login
- Google Login
- JWT Authentication
- Protected Routes
- Account Linking
- MongoDB Database
- Express.js Backend
- React Frontend Support

A user can:

- Register using Email & Password
- Login using Email & Password
- Login using Google
- Use both methods with the same account

Project Structure

```
src/
│
├── config/
│   └── db.js
│
├── models/
│   └── user.model.js
│
├── controllers/
│   └── auth.controller.js
│
├── middleware/
│   └── auth.middleware.js
│
```

```
├── routes/
│   └── auth.routes.js
│
├── app.js
└── server.js
```

Required Packages

```
npm install express mongoose bcryptjs jsonwebtoken dotenv cors google-auth-library
```

Environment Variables

Create .env

```
PORT=5000

MONGO_URI=mongodb://127.0.0.1:27017/auth_db

JWT_SECRET=mySuperSecretKey

GOOGLE_CLIENT_ID=your_google_client_id
```

Database Connection

config/db.js

```
import mongoose from "mongoose";

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);

    console.log("Database Connected");
  } catch (error) {
    console.log(error);
    process.exit(1);
  }
};

export default connectDB;
```

User Model

models/user.model.js

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
    },

    email: {
      type: String,
      unique: true,
      required: true,
    },

    password: {
      type: String,
      default: null,
    },

    googleId: {
      type: String,
      default: null,
    },

    avatar: {
      type: String,
      default: "",
    },
  },
  {
    timestamps: true,
  }
);

export default mongoose.model("User", userSchema);
```

Auth Controller

controllers/auth.controller.js

```
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import User from "../models/user.model.js";
import { OAuth2Client } from "google-auth-library";

const client = new OAuth2Client(
  process.env.GOOGLE_CLIENT_ID
);

const generateToken = (userId) => {
  return jwt.sign(
    {
      userId,
    },
    process.env.JWT_SECRET,
    {
      expiresIn: "7d",
    }
  );
};

export const register = async (req, res) => {
  try {
    const { name, email, password } = req.body;

    const exists = await User.findOne({ email });

    if (exists) {
      return res.status(400).json({
        success: false,
        message: "User already exists",
      });
    }

    const hashedPassword =
      await bcrypt.hash(password, 10);

    const user = await User.create({
      name,
      email,
      password: hashedPassword,
    });

    const token = generateToken(user._id);

    res.status(201).json({
      success: true,
      token,
      user,
```

```
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message,
    });
  }
};

export const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });

    if (!user) {
      return res.status(400).json({
        success: false,
        message: "Invalid credentials",
      });
    }

    if (!user.password) {
      return res.status(400).json({
        success: false,
        message:
          "This account uses Google Login",
      });
    }

    const isMatch = await bcrypt.compare(
      password,
      user.password
    );

    if (!isMatch) {
      return res.status(400).json({
        success: false,
        message: "Invalid credentials",
      });
    }

    const token = generateToken(user._id);

    res.status(200).json({
      success: true,
      token,
      user,
    });
```

```
  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message,
    });
  }
};

export const googleLogin = async (
  req,
  res
) => {
  try {
    const { token } = req.body;

    const ticket =
      await client.verifyIdToken({
        idToken: token,
        audience:
          process.env.GOOGLE_CLIENT_ID,
      });

    const payload = ticket.getPayload();

    const {
      sub,
      email,
      name,
      picture,
    } = payload;

    let user = await User.findOne({
      email,
    });

    if (!user) {
      user = await User.create({
        name,
        email,
        googleId: sub,
        avatar: picture,
      });
    } else {
      if (!user.googleId) {
        user.googleId = sub;
        user.avatar = picture;

        await user.save();
      }
```

```
    }

    const jwtToken = generateToken(
      user._id
    );

    res.status(200).json({
      success: true,
      token: jwtToken,
      user,
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message,
    });
  }
};
```

Authentication Middleware

middleware/auth.middleware.js

```
import jwt from "jsonwebtoken";

export const protect = (
  req,
  res,
  next
) => {
  try {
    const authHeader =
      req.headers.authorization;

    if (!authHeader) {
      return res.status(401).json({
        message: "Unauthorized",
      });
    }

    const token =
      authHeader.split(" ")[1];

    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET
    );
```

```
    req.user = decoded;

    next();
  } catch (error) {
    return res.status(401).json({
      message: "Invalid Token",
    });
  }
};
```

Routes

routes/auth.routes.js

```
import express from "express";

import {
  register,
  login,
  googleLogin,
} from "../controllers/auth.controller.js";

const router = express.Router();

router.post("/register", register);

router.post("/login", login);

router.post("/google", googleLogin);

export default router;
```

Express App

app.js

```
import express from "express";
import cors from "cors";

import authRoutes from "./routes/auth.routes.js";

const app = express();
```

```
app.use(cors());

app.use(express.json());

app.use(
  "/api/auth",
  authRoutes
);

export default app;
```

Server

server.js

```
import dotenv from "dotenv";

dotenv.config();

import app from "./app.js";

import connectDB from "./config/db.js";

connectDB();

app.listen(
  process.env.PORT,
  () => {
    console.log(
      `Server Running On Port ${process.env.PORT}`
    );
  }
);
```

API Endpoints

Register

POST

`/api/auth/register`

Body

```
{
  "name": "John",
  "email": "john@gmail.com",
  "password": "123456"
}
```

Login

POST

`/api/auth/login`

Body

```
{
  "email": "john@gmail.com",
  "password": "123456"
}
```

Google Login

POST

`/api/auth/google`

Body

```
{
  "token": "google_id_token"
}
```

Frontend Google Login

Install

```
npm install @react-oauth/google
```

App.jsx

```
import {
  GoogleOAuthProvider,
} from "@react-oauth/google";

<GoogleOAuthProvider
  clientId={GOOGLE_CLIENT_ID}
>
```

```
  <App />
</GoogleOAuthProvider>;
```

Login Button

```
import {
  GoogleLogin,
} from "@react-oauth/google";

import axios from "axios";

const handleSuccess = async (
  response
) => {
  await axios.post(
    "http://localhost:5000/api/auth/google",
    {
      token:
        response.credential,
    }
  );
};

<GoogleLogin
  onSuccess={handleSuccess}
/>;
```

Authentication Flow

Email Login Flow

```
Register
↓
Hash Password
↓
Save User
↓
Generate JWT
↓
Login
↓
JWT Token
```

Google Login Flow

```
Google Sign In
↓
Google Token
```

```
↓
Backend Verification
↓
Find/Create User
↓
Generate JWT
↓
Login Success
```

Account Linking Flow

```
Email Account Exists
↓
User Clicks Google Login
↓
Same Email Found
↓
Attach Google ID
↓
Single Account
```